

# Massively Parallel Computation of Matching and MIS in Sparse Graphs

Soheil Behnezhad\*  
University of Maryland  
soheil@cs.umd.edu

Sebastian Brandt  
ETH Zurich  
brandts@ethz.ch

Mahsa Derakhshan\*  
University of Maryland  
mahsa@cs.umd.edu

Manuela Fischer  
ETH Zurich  
manuela.fischer@inf.ethz.ch

MohammadTaghi Hajiaghayi\*  
University of Maryland  
hajiagha@cs.umd.edu

Richard M. Karp  
UC Berkeley  
karp@cs.berkeley.edu

Jara Uitto†  
ETH Zurich & University of Freiburg  
jara.uitto@inf.ethz.ch

## ABSTRACT

The *Massively Parallel Computation* (MPC) model serves as a common abstraction of many modern large-scale parallel computation frameworks and has recently gained a lot of importance, especially in the context of classic graph problems. In this work, we mainly consider *maximal matching* and *maximal independent set* problems in the MPC model.

These problems are known to admit efficient MPC algorithms if the space available per machine is near-linear in the number  $n$  of nodes. This is not only often significantly more than what we can afford, but also allows for easy if not trivial solutions for sparse graphs—which are common in real-world large-scale graphs. We are, therefore, interested in the *low-memory* MPC model, where the space per machine is restricted to be strongly sublinear, that is,  $n^\delta$  for any constant  $0 < \delta < 1$ .

We parametrize our algorithms by the *arboricity*  $\lambda$  of the input graph. Our key ingredient is a degree reduction technique that reduces these problems in graphs with arboricity  $\lambda$  to the corresponding problems in graphs with maximum degree  $\text{poly}(\lambda, \log n)$  in  $O(\log^2 \log n)$  rounds, giving rise to  $O(\sqrt{\log \lambda} \cdot \log \log \lambda + \log^2 \log n)$ -round algorithms.

Our result is particularly interesting for graphs with  $\text{poly} \log n$  arboricity as for such graphs, we get  $O(\log^2 \log n)$ -round algorithms. This covers most natural families of sparse graphs and almost exponentially improves over previous algorithms that all required  $\log^{\Omega(1)} n$  rounds in this regime of MPC.

Finally, our maximal matching algorithm can be employed to obtain a  $(1 + \epsilon)$ -approximate *maximum cardinality matching*, a  $(2 + \epsilon)$ -approximate *maximum weighted matching*, as well as a 2-approximate *minimum vertex cover* in essentially the same number of rounds.

## CCS CONCEPTS

• **Theory of computation** → **Parallel algorithms; Massively parallel algorithms.**

### ACM Reference Format:

Soheil Behnezhad, Sebastian Brandt, Mahsa Derakhshan, Manuela Fischer, MohammadTaghi Hajiaghayi, Richard M. Karp, and Jara Uitto. 2019. Massively Parallel Computation of Matching and MIS in Sparse Graphs. In *2019 ACM Symposium on Principles of Distributed Computing (PODC '19)*, July 29–August 2, 2019, Toronto, ON, Canada. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3293611.3331609>

## 1 INTRODUCTION

Due to the prevalence of huge networks, scalable algorithms for fundamental graph problems have gained a lot of importance in the area of parallel computing. The *Massively Parallel Computation* (MPC) model [1, 6, 7, 10, 17, 20] constitutes a common abstraction of several popular large-scale computation frameworks—such as MapReduce [11], Dryad [19], Spark [33], and Hadoop [32]—and thus serves as the basis for the systematic study of massively parallel algorithms.

Most classic parallel or distributed algorithms can be simulated directly in the MPC model in the same number of rounds (often polylogarithmic) [17, 20]. The additional power of MPC, however, in free local computation (compared to PRAM) or of global all-to-all communication (compared to LOCAL) could potentially be exploited to obtain faster MPC algorithms. Achieving such improvements has been the main theme of research over the past few years in the study of this model.

Surprisingly, for *maximal matching* (MM) and *maximal independent set* (MIS) problems, which have been studied from the very early days of the area in 1980s, we are still far from understanding the full power of MPC. A remarkable series of recent results[2, 10, 14] show how the local memory can be reduced to

\*Supported in part by NSF grants CCF:SPX 1822738, IIS:BIGDATA 1546108, DARPA grant SI3CMD, UMD Year of Data Science Program Grant, and Northrop Grumman Faculty Award.

†Supported by ERC Grant No. 336495 (ACDC).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

PODC '19, July 29–August 2, 2019, Toronto, ON, Canada

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6217-7/19/07...\$15.00

<https://doi.org/10.1145/3293611.3331609>

be  $\tilde{O}(n)$  while still having  $\text{poly}(\log \log n)$ -round algorithms. Yet, all these algorithms fail to go (substantially) below linear space without an (almost) exponential blow-up in the round complexity. Even more severely, for maximal matching, the only subpolylogarithmic algorithm by Lattanzi, Moseley, Suri, and Vassilvitskii [22] requires the memory per machine to be substantially superlinear in the number  $n$  of nodes in the graph.

The space requirement of the aforementioned algorithms is not only prohibitively large, but also allows an easy or even trivial solution for sparse graphs. This is unfortunate, since sparse large-scale graphs indeed appear quite a lot in practice [5, 10, 20]. For this, we study the *low-memory* MPC model with strongly sublinear space. It is natural to ask:

*Can we take advantage of the graphs' sparsity to design subpolylogarithmic, and ideally,  $\text{poly}(\log \log n)$ -round algorithms in the truly sublinear regime of the MPC model?*

We make a step towards answering this question by parametrizing our algorithms by *arboricity*<sup>1</sup> of the input graph. Arboricity is a well-received measure of sparsity that does not impose strict structural constraints such as planarity, bounds on maximum degree, or the like [3, 4, 9, 12, 18]. The family of graphs with arboricity  $\text{poly}(\log n)$ —often called *uniformly sparse* graphs, and also known as *sparse everywhere* graphs—includes but is not restricted to graphs with maximum degree  $\text{poly}(\log n)$ , minor-closed graphs (e.g., planar graphs and graphs with bounded treewidth), as well as preferential attachment graphs, and thus arguably contains most sparse graphs of practical relevance [16, 29]. We show that:

**Main Result.** *There exist low-memory MPC algorithms that with high probability compute a maximal matching and maximal independent set of any given  $n$ -vertex graph of arboricity up to  $\lambda$ , in  $O(\sqrt{\log \lambda} \cdot \log \log \lambda + \log^2 \log n)$  rounds.*

For all graphs with arboricity up to  $\text{poly}(\log n)$ , our algorithms take only  $O(\log^2 \log n)$  rounds to find maximal matching or maximal independent set. This is almost exponentially faster than the previous algorithms in the low-memory MPC model, which we overview soon.

## 1.1 Low-Memory MPC Model for Graph Problems

We have  $M$  machines with local memory of  $S = O(n^\delta)$  words each, for some  $0 < \delta \leq 1^2$ . A graph with  $n$  nodes,  $m$  edges, and maximum degree  $\Delta$  is distributed arbitrarily across the machines. We assume the total memory in the system to be (nearly) linear in the input, i.e.,  $M \cdot S = \tilde{O}(m)$ . The computation proceeds in rounds consisting of *local computation* in all machines in parallel, followed by *global communication* between the machines. We require that the total size of sent and received messages of a machine in every communication

phase does not exceed its local memory capacity. The main interest lies in minimizing the number of rounds, aiming for  $\text{poly}(\log \log n)$ .

## 1.2 Our Contribution

The main technicality of our approach lies in a degree reduction technique that reduces the problems of maximal matching and maximal independent set in a graph with arboricity  $\lambda$  to the corresponding problems in graphs with maximum degree  $\text{poly}(\lambda)$  in  $O(\log^2 \log n)$  rounds.

**THEOREM 1.1.** *There is an  $O(\log \log_\Delta n \cdot \log \log_\Delta \Delta)$ -round low-memory MPC algorithm that w.h.p.<sup>3</sup> reduces maximal matching and maximal independent set in graphs with arboricity  $\lambda = n^{o(1)}$  to the respective problems in graphs with maximum degree  $O(\lambda^{20} + \log^{20} n)$ .<sup>4</sup>*

This improves over the degree reduction algorithm by Barenboim, Elkin, Pettie, and Schneider [4, Theorem 7.2], which runs in  $O(\log_\lambda n)$  rounds in the LOCAL model and can be straightforwardly implemented in the low-memory MPC model.

Our degree reduction technique, combined with the state-of-the-art algorithms for maximal matching and maximal independent set by Ghaffari and Uitto [15] applied as a black box, gives rise to a number of low-memory MPC algorithms, as overviewed next.

**THEOREM 1.2.** *There is an  $O(\sqrt{\log \lambda} \cdot \log \log \lambda + \log \log n \cdot \log \log \Delta)$ -round low-memory MPC algorithm that w.h.p. computes a maximal matching in a graph with arboricity  $\lambda$ .*

This improves over the  $O(\log \lambda + \sqrt{\log n})$ -round LOCAL algorithm by [4] and the  $\tilde{O}(\sqrt{\log \Delta})$ -round algorithm by [15]. The only previous subpolylogarithmic round maximal matching algorithm due to Lattanzi et al. [22], requires space per machine of  $n^{1+\Omega(1)}$ .

We also note that the round complexity of all previous  $\text{poly}(\log \log n)$ -round MPC matching approximation algorithms for general graphs (even if we allow linear memory) [2, 10, 14] blows up by an  $\Omega(\log n)$  overhead for the case of maximal matching. In fact, the problem of finding a maximal matching seems to be much more difficult than finding a  $(1 + \epsilon)$ -approximate maximum matching. Indeed, currently, an  $O(1)$ -approximation can be found almost exponentially faster than a maximal matching, and the approximation ratio can be easily improved from any constant to  $1 + \epsilon$  using a reduction of McGregor [26].

**THEOREM 1.3.** *There is an  $O(\sqrt{\log \lambda} \cdot \log \log \lambda + \log \log n \cdot \log \log \Delta)$ -round low-memory MPC algorithm that w.h.p. computes a maximal independent set in a graph with arboricity  $\lambda$ .*

This algorithm improves over the  $O(\log \lambda + \sqrt{\log n})$ -round algorithm that is obtained by simulating the LOCAL algorithm of [4, 13] and over the  $\tilde{O}(\sqrt{\log \Delta})$ -round algorithm of Ghaffari and Uitto [15]. It also improves over another  $O(\log^3 \log n)$  round algorithm for MIS on trees by Brandt, Fischer and Uitto [8].

<sup>1</sup>The arboricity of a graph is the minimum number of forests into which its edges can be partitioned. Equivalently, Nash-Williams [28] showed that it can be defined as the average degree within the graph's densest subgraph.

<sup>2</sup>Note that we do not require  $\delta$  to be a constant. For the sake of simplicity of presentation, we decided to omit the  $\delta$ -dependency, which is a multiplicative factor of  $O(\frac{1}{\delta})$ , in the analysis of the running time of our algorithms.

<sup>3</sup>As usual, w.h.p. stands for *with high probability*, and means with probability at least  $1 - n^{-c}$  for any constant  $c \geq 1$ .

<sup>4</sup>The purpose of the choice of all the constants in this work is merely to simplify presentation.

As a maximal matching automatically provides 2-approximations for maximum matching and minimum vertex cover, Theorem 1.2 directly implies the following result.

**Corollary 1.4.** *There is an  $O(\sqrt{\log \lambda} \cdot \log \log \lambda + \log \log n \cdot \log \log \Delta)$ -round low-memory MPC algorithm that w.h.p. computes a 2-approximate maximum matching and a 2-approximate minimum vertex cover in a graph with arboricity  $\lambda$ .*

McGregor's reduction [26] allows us to further improve the approximation to  $1 + \varepsilon$ .

**Corollary 1.5.** *There is an  $O((\frac{1}{\varepsilon})^{O(1/\varepsilon)} \cdot (\sqrt{\log \lambda} \cdot \log \log \lambda + \log \log n \cdot \log \log \Delta))$ -round low-memory MPC algorithm that w.h.p. computes a  $(1 + \varepsilon)$ -approximate maximum matching, for any  $\varepsilon > 0$ .*

Due to a reduction by Lotker, Patt-Shamir, and Pettie [25], our constant-approximate matching algorithm can also be employed to find a  $(2 + \varepsilon)$ -approximate maximum weighted matching.

**Corollary 1.6.** *There is an  $O((\frac{1}{\varepsilon})^{O(1/\varepsilon)} \cdot (\sqrt{\log \lambda} \cdot \log \log \lambda + \log \log n \cdot \log \log \Delta))$ -round low-memory MPC algorithm that w.h.p. computes a  $(2 + \varepsilon)$ -approximate maximum weighted matching, for any  $\varepsilon > 0$ .*

## 2 ALGORITHM OUTLINE AND ROADMAP

In the low-memory setting, one is inevitably confronted with the challenge of locality: as the space of a machine is strongly sublinear, it will never be able to see a significant fraction of the nodes, regardless of how sparse the graph is. Further building on the ideas by [8], we cope with this imposed locality by adopting local techniques—mainly inspired by the LOCAL model [24]—and enhancing them with the additional power of global communication, in order to achieve an improvement in the round complexity compared to the LOCAL algorithms while still being able to guarantee applicability in the presence of strongly sublinear memory.

The main observation behind our algorithms is the following: If the maximum degree in the graph is small, LOCAL algorithms can be simulated efficiently in the low-memory MPC model. Our method thus basically boils down to reducing the maximum degree of the input graph, as described in Theorem 1.1, and correspondingly consists of two parts: a *degree reduction* part followed by a *LOCAL simulation* part. In the degree reduction part, which constitutes the key ingredient of our algorithm, we want to find a partial solution (that is, either a matching or an independent set) so that the *remainder graph*—i.e., the graph after the removal of this partial solution (that is, after removing all matched nodes or after removing the independent set nodes along with all their neighbors)—has smaller degree.

**Lemma 2.1.** *There are  $O(\log \log n \cdot \log \log \Delta)$ -round low-memory MPC algorithms that compute a matching and an independent set in a graph with arboricity  $\lambda = n^{o(1)}$  so that the remainder graph w.h.p. has maximum degree  $O((\max\{\lambda, \log n\})^{20})$ .*

Note that this directly implies Theorem 1.1. Next, we show how Theorems 1.2 and 1.3 follow from Lemma 2.1 as well as from an efficient simulation of LOCAL algorithms due to [15].

**PROOF OF THEOREMS 1.2 AND 1.3.** If  $\lambda$ , and hence  $\Delta$ , is at least polynomial in  $n$ , we directly apply the algorithm by [15], which runs

in  $O(\sqrt{\log \Delta} \cdot \log \log \Delta + \sqrt{\log \log n}) = O(\sqrt{\log n} \cdot \log \log n)$  rounds. Otherwise, we first apply the algorithm of Lemma 2.1 to obtain a partial solution that reduces the degree in the remainder graph to  $\Delta' = O(\lambda^{20})$  if  $\lambda \geq \log n$ , or to  $\Delta' = O(\log^{20} n)$  if  $\lambda \leq \log n$ . It runs in  $O(\log \log n \cdot \log \log \Delta)$  rounds. We then apply the algorithm by [15] on the remainder graph. This takes  $O(\sqrt{\log \Delta'} \cdot \log \log \Delta' + \sqrt{\log \log n}) = O(\sqrt{\log \lambda} \cdot \log \log \lambda + \sqrt{\log \log n})$  rounds.  $\square$

Our degree reduction algorithm in Lemma 2.1 consists of several phases, each reducing the maximum degree by a polynomial factor, as long as the degree is still large enough.

**Lemma 2.2.** *There are  $O(\log \log n)$ -round low-memory MPC algorithms that compute a matching and an independent set, respectively, in a graph with arboricity  $\lambda = o(\text{poly}(n))$  and maximum degree  $\Delta = \Omega((\max\{\lambda, \log n\})^{20})$  so that the remainder graph w.h.p. has maximum degree  $O(\Delta^{0.4})$ .*

We first show that indeed iterated applications of this polynomial degree reduction lead to the desired degree reduction in Lemma 2.1.

**PROOF OF LEMMA 2.1.** We iteratively apply the polynomial degree reduction from Lemma 2.2, observing that as long as the maximum degree is still in  $\Omega(\lambda^{20})$  and  $\Omega(\log^{20} n)$ , we reduce the maximum degree by a polynomial factor from  $\Delta$  to  $O(\Delta^{0.4})$  in each phase, resulting in at most  $O(\log \log \Delta)$  phases.  $\square$

It remains to show that such a polynomial degree reduction, as claimed in Lemma 2.2, indeed is possible. This is done in two parts. First, in Section 3, we provide a centralized algorithm for a polynomial degree reduction, and then, in Section 4, we show how to implement this centralized algorithm efficiently in the low-memory MPC model. Several of the techniques we use throughout the paper can also be applied in a wider context. To provide some algorithmic tools for future work, we introduce generalized versions of a selection of our techniques in Section 5.

**Remark 2.3.** *While our algorithms, at first sight, seem to need to know  $\lambda$ , we can easily get rid of this assumption by employing the standard technique [21] of running the algorithm with doubly-exponentially increasing estimates for  $\lambda$ . Roughly speaking, the idea is as follows. We run  $O(\log \log \Delta)$  copies, one for every estimate of  $\lambda$ , of the algorithm in parallel and pick (one of) the correct solution(s), which is easy to find in MPC.*

## 3 A CENTRALIZED DEGREE REDUCTION ALGORITHM

In this section, we present a centralized algorithm for the polynomial degree reduction as stated in Lemma 2.2. For details on how this algorithm can be implemented in the low-memory MPC model, we refer to Section 4. In Section 3.1, we give a formal description of the (centralized) algorithm. Then, in Section 3.2, we prove that this algorithm indeed leads to a polynomial degree reduction.

### 3.1 Algorithm Description

In the following, we set  $d = \Delta^{1/10}$ , and observe that  $d = \Omega(\lambda^2)$  as well as  $d = \Omega(\log^2 n)$ , due to the assumptions on  $\Delta$  in the lemma statement. We present an algorithm that reduces the maximum

degree to  $O(d^4)$ . This algorithm consists of three phases: a *partition* phase, in which the nodes are partitioned into layers so that every node has at most  $d$  neighbors in higher-index layers, a *mark-and-propose* phase in which a random set of candidates is proposed independently in every layer, and a *selection* phase in which a valid subset of the candidate set is selected as partial solution by resolving potential conflicts across layers.

**Partition Phase:** We compute an  $H$ -partition, that is, a partition of the nodes into layers so that every node has at most  $d$  neighbors in layers with higher (or equal) index [3, 27, 28].

**Definition 3.1 ( $H$ -Partition).** An  $H$ -partition with out-degree  $d$ , defined for any  $d > 2\lambda$ , is a partition of the nodes into  $\ell$  layers  $L_1, \dots, L_\ell$  with the property that a node  $v \in L_i$  has at most  $d$  neighbors in  $\bigcup_{j=i}^\ell L_j$ . We call  $i$  the layer index of  $v$  if  $v \in L_i$ . For neighbors  $u \in L_i$  and  $v \in L_j$  for  $i < j$ , we call  $v$  a parent of  $u$  and  $u$  a child of  $v$ . If we think of the edges as being directed from children to parents, this gives rise to a partial orientation of the edges, with no orientation of the edges connecting nodes in the same layer.

Note that, for  $d > 2\lambda$ , such a partition can be computed easily by the following sequential greedy algorithm, also known as *peeling* algorithm: Iteratively, for  $i \geq 1$ , put all remaining nodes with remaining degree at most  $d$  into layer  $i$ , and remove them from the graph. Due to the well-known fact that the average degree of a graph with arboricity  $\lambda$  is  $2\lambda$ , in every iteration a constant fraction of the nodes will be removed.

**Mark-and-Propose Phase:** We first mark a random set of candidates (either edges or nodes) and then propose a subset of these marked candidates for the partial solution as follows.

In the case of maximal matching, every node first marks an outgoing edge chosen uniformly at random and then proposes one of its incoming marked edges, if any, uniformly at random.

In the case of maximal independent set, every node marks itself independently with probability  $p = d^{-2}$ . Then, if a node is marked and none of its neighbors in the same layer is marked, this node is proposed. Note that whether a marked node gets proposed only depends on nodes in the same layer, thus on neighbors with respect to unoriented edges.

**Selection Phase:** The set of proposed candidates might not be a valid solution, meaning that it might have some conflicts (i.e., two incident edges or two neighboring nodes). In the selection phase, possible conflicts are resolved (deterministically) by picking an appropriate subset of the proposed candidates, as follows. Iteratively, for  $i = \ell, \dots, 1$ , all (remaining) proposed candidates in layer  $i$  are added to the partial solution and then removed from the graph.

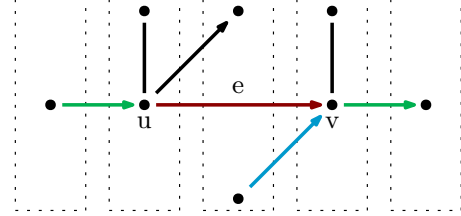
In the case of maximal matching, we add all (remaining) proposed edges directed to a node in layer  $i$  to the matching and remove both their endpoints from the graph.

In the case of maximal independent set, we add all (remaining) proposed nodes in layer  $i$  to the independent set and remove them along with their neighbors from the graph.

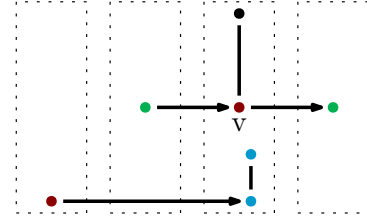
### 3.2 Proof of Correctness

It is easy to see that the selected solution is a valid partial solution, that is, that there are no conflicts. It remains to be shown that the

**Figure 1: Illustration of the mark-and-propose and the selection phase for matching in (a) and independent set in (b). Blue indicates marked but not proposed, green stands for (marked and) proposed but not selected, and red means (marked and proposed and) selected. Note that we omitted all (but a few) irrelevant edges from the figure; the partition into layers thus might not correspond to a valid  $H$ -partition.**



(a) An (oriented) edge  $e = (u, v)$  that is selected to be added to the matching cannot have an incident edge that is also selected: an unoriented incident edge cannot be marked as only oriented edges are marked; an oriented edge with the same starting point  $u$  cannot be marked as  $u$  marks only one outgoing edge; an oriented edge with the same end-point  $v$  cannot be proposed as  $v$  proposes only one incoming edge; all other oriented edges  $f$  are either processed before (in the case of an outgoing edge from  $v$ ) or after (in the case of an incoming edge to  $u$ ) edge  $e$  in the selection phase. In the former case, the selection of  $f$  would lead to the removal of  $e$  before  $e$  is processed;  $e$  thus would not be selected. In the latter case, the edge  $f$  is removed immediately after  $e$  is selected (and thus before  $f$  is processed), and thus cannot be selected.



(b) If two neighboring nodes are marked, none of them will be proposed, and consequently, none of them will be selected. A node  $v$  that is selected to be added to the independent set cannot have a neighbor that is also selected: a neighbor in the same layer cannot be marked as otherwise  $v$  would not be proposed; a neighbor in a lower-index layer is removed from the graph when  $v$  joins the independent set, and hence before it potentially could be selected; a selected neighbor in a higher-index layer would lead to  $v$ 's immediate removal from the graph; when  $v$ 's layer is processed,  $v$  is not part of the graph anymore, and thus could not be selected.

degree indeed drops to  $O(d^4)$ . As the out-degree is bounded by  $d$ , it is enough to show the following.

**Lemma 3.2.** Every node with in-degree at least  $d^4$  gets removed or all but  $d^4$  of its incoming edges get removed, with high probability.

**PROOF OF LEMMA 3.2 FOR MATCHING.** Let  $v$  be a node with degree at least  $d^4$ . First, observe that if at least one incoming edge of  $v$  is proposed, then an edge incident to  $v$  (not necessarily incoming) will be selected to be added to the matching. This is because the only reason why  $v$  would not select a proposed incoming edge is that  $v$  has already been removed from the graph, and this happens only if its proposed edge has been selected to be added to

the matching in a previous step. It thus remains to show that every node  $v$  with in-degree at least  $d^4$  with high probability will have at least one incoming edge that has been proposed by the respective child. As every incoming edge of  $v$  is marked independently with probability at least  $1/d$ , the probability of  $v$  not having a marked, and hence not having a proposed incoming edge is at most  $(1 - 1/d)^{d^4} \leq e^{-d^3} = e^{-\Omega(\log^6 n)} = o\left(\frac{1}{\text{poly}(n)}\right)$ . A union bound over all nodes with degree at least  $d^4$  concludes the proof.  $\square$

**PROOF OF LEMMA 3.2 FOR INDEPENDENT SET.** Let  $v$  be a node in layer  $i$  that is still in the graph and has at least  $d^4$  children after all layers with index  $\geq i$  have been processed. We show that then at least one of these children will be selected to join the independent set with high probability. Note that this then concludes the proof, as in all the cases either  $v$  will be removed from the graph or will not have a high in-degree anymore. Moreover, observe that such a child  $u$  of  $v$  (that is still there after having processed layers  $\geq i$ ) will be selected to join the independent set iff it is proposed. This is because if it did not join even though it is proposed, then a parent of  $u$  would have been selected to join the independent set, in which case  $u$  would not have been part of the graph anymore, at latest after  $i$  has been processed, thus would not count towards  $v$ 's high degree at that point.

Every such child  $u$  of  $v$  is marked independently with probability  $p = d^{-2}$ . The probability of  $u$  being proposed and hence joining the independent set is at least  $p(1-p)^d$ , as it has at most  $d$  neighbors in its layer, and it is proposed iff it is marked and none of its neighbors in the same layer is marked. Node  $v$  thus in expectation has at least  $\mu := d^4 p(1-p)^d \geq d^2 e^{-2/d} = \Omega(d^2)$  children that join the independent set.

Since whether a node  $u$  proposes and hence joins the independent set depends on at most  $d$  other nodes (namely  $u$ 's neighbors in the same layer), it thus follows from a variant of the Chernoff bounds for bounded dependence, e.g., from Theorem 2.1 in [31], that the probability of  $v$  having, say,  $0.5\mu$  neighbors that join the independent set is at most  $e^{-\Omega(\mu/d)} = e^{-\Omega(d)} \leq e^{-\Omega(\log^2 n)} = o\left(\frac{1}{\text{poly}(n)}\right)$ . A union bound over all nodes  $v$  with degree at least  $d^4$  concludes the proof.  $\square$

## 4 IMPLEMENTATION OF THE DEGREE REDUCTION ALGORITHM IN MPC

The degree reduction algorithm described in Section 3 can be implemented in the LOCAL model in a relatively straightforward way in  $\text{poly}(\log n)$  rounds. Due to the nature of LOCAL, this implies that every node/edge in the input graph can decide on its output by learning the topology of its  $\text{poly}(\log n)$ -hop neighborhood. Given the additional power of the global all-to-all communication in the MPC model, however, one could hope for much faster algorithms than in the LOCAL model.

**Graph Exponentiation:** A very natural and intuitive approach for a speed-up in the all-to-all communication setting is the well-known *graph exponentiation* technique [23], which can be summarized as follows: Suppose that every node knows its  $2^{i-1}$ -hop neighborhood in iteration  $i - 1$ . Then, in iteration  $i$ , each node can inform the nodes in its  $2^{i-1}$ -hop neighborhood of the topology of its  $2^{i-1}$ -hop

neighborhood. Hence, every node can learn of its  $2^i$ -hop neighborhood in iteration  $i$ , allowing it to simulate any  $2^i$ -round LOCAL algorithm in 0 rounds. Using this exponentiation technique, in principle, it is conceivable that every  $t$ -round LOCAL algorithm can be simulated in  $O(\log t)$  MPC rounds. However, the memory restrictions in the truly sublinear regime of MPC impose the following two fundamental barriers for such an approach:

**Local memory barrier.** For any  $t \geq 1$ , the  $t$ -hop neighborhood of a node may be as large as  $\Omega(m)$ , exceeding the local memory of a machine. Even the 1-hop neighborhood of a node with degree higher than  $\omega(n^\delta)$  cannot be stored on a single machine.

**Global memory barrier.** Storing the neighborhood of each node on its corresponding machine leads to storing overlapping neighborhoods, and hence redundant copies of nodes/edges, and thus a total aggregated memory that is significantly larger than the input size  $m$ .

In order to deal with these issues, the key observation is that a large fraction of the nodes in the graph are contained in the first layers of the  $H$ -partition. In particular, we show that if we focus on the graph remaining after  $\ell/2$  iterations of peeling, we can perform roughly  $\log \ell$  exponentiation steps without violating the memory constraints. Hence, we can perform  $2^i$  steps of the degree reduction process in roughly  $i$  communication rounds. Before going into the details of the simulation, we note that the simulation approach introduced in this section is tailor-made for the MIS and MM problems. In Section 5.3, we show that the ideas used in the simulation generalize beyond MIS and MM. However, our generalization comes with a slight cost in the global memory requirement.

### 4.1 Distributed Simulation of the Centralized Degree Reduction

In this section, we show how to simulate the centralized degree reduction algorithm from Section 3 in the low-memory MPC model. In Section 4.2, we show how to implement the partition phase efficiently in the low-memory MPC model. Then, in Section 4.3, we describe how to perform the simulation of the mark-and-propose as well as the selection phase. Together with the correctness proof established in Section 3.2, this will conclude the proof of Lemma 2.2.

Throughout our algorithm, we often have to compute functions that depend on all the neighbors of some node  $v$ , such as the degree of  $v$ . The problem with computing such functions is that the degree of a vertex might be larger than the memory per machine.

We define a special class of such functions, which we call *separable*<sup>5</sup>, that can be parallelized very efficiently. Due to their verbosity, we defer the formal lemma statements and their proofs to Section 5.2. For the rest of the section, we simply assume that any separable function can be computed in constant time.

**Definition 4.1.** Let  $f : 2^{\mathbb{R}} \rightarrow \mathbb{R}$  denote a set function. The function  $f$  is separable iff for any set of reals  $A$  and for any  $B \subseteq A$ , we have  $f(A) = f(f(B), f(A \setminus B))$ .

**Remark 4.2.** Notice that for the peeling, we need to compute the degree of every remaining node in every iteration. Here, we can use the

<sup>5</sup>We note that the definition is almost the same of a *MRC-parallelizable* function by Karloff et al. [20]. Ours is slightly cleaner and yet sufficient for our purposes.

fact that computing the degree is separable, and thus avoid problems in the case where the degree of a node is too large to fit the local memory of a single machine.

## 4.2 Partition Phase

We first prove some properties of the  $H$ -partition constructed by the greedy peeling algorithm that will be useful for an efficient implementation in the low-memory MPC model.

**Lemma 4.3.** *The  $H$ -partition with out-degree  $d$ , constructed by the greedy peeling algorithm, satisfies the following properties.*

- (i) *For all  $0 \leq i \leq \ell$ , the number  $\left| \bigcup_{j=i}^{\ell} L_j \right|$  of nodes in layers with index  $\geq i$  is at most  $n \left( \frac{2\lambda}{d} \right)^{i-1}$ . In other words, if we remove all nodes in layer  $i$  from the set of nodes in layers  $\geq i$ , then the number of nodes drops by a factor of  $\frac{2\lambda}{d}$ , i.e.,  $\left| \bigcup_{j=i+1}^{\ell} L_j \right| \leq \frac{2\lambda}{d} \left| \bigcup_{j=i}^{\ell} L_j \right|$  for all  $0 \leq i \leq \ell$ .*
- (ii) *There are at most  $\ell = O\left(\log_{\frac{d}{\lambda}} n\right)$  layers.*

**PROOF.** We prove (i) by induction, thus assume that there are  $n_i \leq n \left( \frac{2\lambda}{d} \right)^{i-1}$  nodes in the graph  $H_i$  induced by nodes in layers  $\geq i$ . Towards a contradiction, suppose that there are  $n_{i+1} > n \left( \frac{2\lambda}{d} \right)^i$  nodes in layers  $\geq i+1$ . By construction, all these nodes must have had degree larger than  $d$  in  $H_i$ , as otherwise they would have been added to layer  $i$ . This results in an average degree of more than  $\frac{n_{i+1}d}{n_i} = 2\lambda$  in  $H_i$ , which contradicts the well-known upper bound of  $2\lambda$  on the average degree in a graph that has arboricity at most  $\lambda$ . Note that (ii) is a direct consequence of (i).  $\square$

In the following, we describe how to compute the  $H$ -partition with parameter  $d = \Delta^{1/10}$  in the low-memory MPC model. Throughout this section, we assume that  $\Delta \geq (2\lambda)^{20}$ , i.e., that  $d \geq (2\lambda)^2$ . Observe that if  $\Delta^2 > n^\delta$ , then the  $H$ -partition with parameter  $d = \Delta^{1/10}$  consists of  $O(\log_{d/\lambda} n) = O(\log_\Delta n) = O(1/\delta)$  layers, in which case the arguments in this section imply that going through the layers one by one will easily yield at least as good runtimes as for the more difficult case of  $\Delta^2 \leq n^\delta$ . Hence, throughout this section, we assume that  $\Delta^2 \leq n^\delta$ .

The goal of the algorithm for computing the  $H$ -partition is that each node (or, more formally, the machine storing the node) knows in which layer of the  $H$ -partition it is contained. The algorithm proceeds in iterations, where each iteration consists of two parts: first, the output, i.e., the layer index, is determined for a large fraction of the nodes, and second, these nodes are removed for the remainder of the computation. The latter ensures that the remaining small fraction of nodes can use essentially all of the total available memory in the next iteration, resulting in a larger memory budget per node. However, there is a caveat: When the memory budget per node exceeds  $\Theta(n^\delta)$ , i.e., the memory capacity of a single machine, then it is not sufficient anymore to merely argue that the used memory of all nodes together does not exceed the total memory of all machines together<sup>6</sup>. We circumvent this issue by starting the above

process repeatedly from anew (in the remaining graph) each time the memory requirement per node reaches the memory capacity of a single machine. As we will see, the number of repetitions, called *phases*, is bounded by  $O(1/\delta)$ .

In the following, we examine the phases and iterations in more detail.

**Algorithm Details:** Let  $k$  be the largest integer s.t.  $\Delta^{2k+1} \leq n^\delta$  (which implies that  $k \geq 0$ ). The algorithm consists of phases and each phase consists of  $k+1$  iterations. Next, we describe our implementation of the graph exponentiation in more detail.

- In each iteration  $i = 0, 1, \dots, k$ , we do the following.
  - Let  $G_i = G_i^{(0)}$  be the graph at the beginning of iteration  $i$ . Each node connects its current 1-hop neighborhood to a clique by adding virtual edges to  $G_i$ ; if  $i = 0$ , omit this step. Perform 20 repetitions of the following process if  $i \geq 1$ , and 60 repetitions if  $i = 0$ :
    - \* In repetition  $0 \leq j \leq 19$  (resp.  $0 \leq j \leq 59$ ), each node computes its layer index in the  $H$ -partition of  $G_i^{(j)}$  (with parameter  $d$ ) or determines that its layer index is strictly larger than  $2^i$ , upon which all nodes in layer at most  $2^i$  (and all its incident edges) are removed from the graph, resulting in a graph  $G_i^{(j+1)}$ .
  - Set  $G_{i+1} = G_i^{(20)}$  (resp.  $G_{i+1} = G_i^{(60)}$  if  $i = 0$ ).

At the end of the phase remove all added edges.

The algorithm terminates when each node knows its layer.

Note that each time a node is removed from the graph, the whole layer that contains this node is removed, and each time such a layer is removed, all layers with smaller index are removed at the same time or before. By the definition of the  $H$ -partition, if we remove the  $\ell$  layers with smallest index from a graph, then there is a 1-to-1 correspondence between the layers of the resulting graph and the layers with layer index at least  $\ell+1$  of the original graph. More specifically, layer  $\ell'$  of the resulting graph contains exactly the same nodes as layer  $\ell + \ell'$  of the original graph. Hence, if a node knows its layer index in some  $G_i^{(j)}$ , it can easily compute its layer index in our original input graph  $G$ , by keeping track of the number of deleted layers, which is uniquely defined by  $i, j$  and the number of the phase. We implicitly assume that each node performs this computation upon determining its layer index in some  $G_i^{(j)}$  and in the following only consider how to determine the layer index in the current graph.

**Implementation in the MPC Model:** Let us take a look at one iteration. Connecting the 1-hop neighborhoods to cliques is done by adding the edges that are missing. Edges that are added by multiple nodes are only added once (since the edge in question is stored by the machines that contain an endpoint of the edge, this is straightforward to realize). Note that during a phase, the 1-hop neighborhoods of the nodes grow in each iteration (if not too many close-by nodes are removed from the graph); more specifically, after  $i$  iterations of connecting 1-hop neighborhoods to cliques, the new

<sup>6</sup>Furthermore, in the “shuffle” step of every MPC round [20], we assume that the nodes are stored in the machines in a balanced way, i.e., as long as a single node fits

onto a single machine and the total memory is not exceeded, the underlying system takes care of load-balancing.

1-hop neighborhood of a node contains exactly the nodes that were contained in its  $2^i$ -hop neighborhood at the beginning of the phase (and were not removed so far).

In iteration  $i$ , the layer of a node is computed as follows: First each node locally gathers the topology of its  $2^i$ -hop neighborhood (without any added edges)<sup>7</sup>. Since this step is performed after connecting the  $2^{i-1}$ -hop neighborhood of each node to a clique (by repeatedly connecting 1-hop neighborhoods to cliques), i.e., after connecting each node to any other node in its  $2^i$ -hop neighborhood, only 1 round of communication is required for gathering the topology. Moreover, since a node that knows the topology of its  $2^i$ -hop neighborhood can simulate any  $(2^i - 1)$ -round distributed process locally, it follows from the definition of the  $H$ -partition, that knowledge of the topology of the  $2^i$ -hop neighborhood is sufficient for a node to determine whether its layer index is at most  $2^i$  and, if this is the case, in exactly which layer it is contained. Thus, the only tasks remaining are to bound the runtime of our algorithm and to show that the memory restrictions of our model are not violated by the algorithm.

**Runtime:** It is easy to see that every iteration takes  $O(1)$  rounds. Thus, in order to bound the runtime of our algorithm, it is sufficient to bound the number of iterations by  $O((1/\delta) \cdot \log \log n)$ . By Lemma 4.3 (ii) the number of layers in the  $H$ -partition of our original input graph  $G$  is  $O(\log_{d/\lambda} n)$ , which is  $O(\log_\Delta n)$  since  $d/(2\lambda) \geq \sqrt{d} = \Delta^{1/20}$ . Consider an arbitrary phase. According to the algorithm description, in iteration  $i \geq 1$ , all nodes in the  $20 \cdot 2^i$  lowest layers are removed from the current graph. Hence, ignoring iteration 0, the number of removed layers doubles in each iteration, and we obtain that the number of layers removed in the  $k + 1$  iterations of our phase is  $\Omega(2^k)$ . By the definition of  $k$ , we have  $\Delta^{2^{k+1}+1} > n^\delta$ , which implies  $2^k > 1/3 \cdot \delta \cdot \log_\Delta n$ .

Combining this inequality with the observations about the total number of layers and the number of layers removed per phase, we see that the algorithm terminates after  $O(1/\delta)$  phases. Since there are  $k + 1 = O(\log \log n)$  iterations per phase, the bound on the number of iterations follows.

**Memory Footprint:** As during the course of the algorithm edges are added and nodes collect the topology of certain neighborhoods, we have to show that adding these edges and collecting these neighborhoods does not violate our memory constraints of  $O(n^\delta)$  per machine. As a first step towards this end, the following lemma bounds the number of nodes contained in graph  $G_i$ .

**Lemma 4.4.** *Consider an arbitrary phase. Graph  $G_i$  from that phase contains at most  $n' / (\Delta^{2^i})$  nodes, for all  $i \geq 1$ , where  $n' = n/\Delta$ .*

**PROOF.** By Lemma 4.3 (i), removing the nodes in the layer with smallest index from the current graph decreases the number of nodes by a factor of at least  $d/(2\lambda) \geq d^{1/2} = \Delta^{1/20}$ . We show the lemma statement by induction. Since in iteration 0 the nodes in the 60 layers with smallest index are removed, we know that  $G_1$  contains at most  $n/(\Delta^3) = n' / (\Delta^{2^1})$  nodes. Now assume that  $G_i$  contains at most  $n' / (\Delta^{2^i})$  nodes, for an arbitrary  $i \geq 1$ . According to

the design of our algorithm,  $G_{i+1}$  is obtained from  $G_i$  by removing the nodes in the  $20 \cdot 2^i$  layers with smallest index. Combining this fact with our observation about the decrease in the number of nodes per removed layer, we obtain that  $G_{i+1}$  contains at most  $n' / (\Delta^{2^i}) \cdot 1 / (\Delta^{2^i}) = n' / (\Delta^{2^{i+1}})$  nodes.  $\square$

Using Lemma 4.4, we now show that the memory constraints of the low-memory MPC model are not violated by our algorithm. Consider an arbitrary phase and an arbitrary iteration  $i$  during that phase. If  $i = 0$ , then no edges are added and each node already knows the topology of its  $2^i$ -hop neighborhood, so no additional memory is required. Hence, assume that  $i \geq 1$ .

Due to Lemma 4.4, the number of nodes considered in iteration  $i$  is at most  $n' / (\Delta^{2^i})$ , where, again,  $n' = n/\Delta$ . After the initial step of connecting 1-hop neighborhoods to cliques in iteration  $i$ , each remaining node is connected to all nodes that were contained in its  $2^i$ -hop neighborhood in the original graph  $G$  (and were not removed so far). Hence, each remaining node is connected to at most  $O(\Delta^{2^i})$  other nodes, resulting in a memory requirement of  $O(\Delta^{2^i})$  per node, or  $O(n/\Delta)$  in total. Similarly, when collecting the topology of its  $2^i$ -hop neighborhood, each node has to store  $O(\Delta^{2^i} \cdot \Delta)$  edges, which requires at most  $O(\Delta^{2^i} \cdot \Delta)$  memory, resulting in a total memory requirement of  $O(n)$ . Hence, the described algorithm does not exceed the total memory available in the low-memory MPC model. Moreover, due to the choice of  $k$ , the memory requirement of each single node does not exceed the memory capacity of a single machine.

### 4.3 Simulation of the Mark-and-Propose and Selection Phase

For the simulation of the mark-and-propose and selection phase, we rely heavily on the approach of Section 4.2. Recall that nodes were removed in *chunks* consisting of several consecutive layers and that before a node  $v$  was removed,  $v$  was directly connected to all nodes contained in a large neighborhood around  $v$  by adding the respective edges. For the simulation, we go through these chunks in the reverse order in which they were removed. Note that in which chunk a node is contained is uniquely determined by the layer index of the node. As each node computes its layer index during the construction of the  $H$ -partition, each node can easily determine in which part of the simulation it will actively participate.

However, there is a problem we need to address: For communication, we would like the edges that we added during the construction of the  $H$ -partition to be available also for the simulation. Unfortunately, during the course of the construction, we removed added edges again to free memory for the adding of other edges. Fortunately, there is a conceptually simple way to circumvent this problem: in the construction of the  $H$ -partition, add a pre-processing step in the beginning, in which we remove the lowest  $c \cdot \log((1/\delta) \cdot \log \log n)$  layers (where  $c$  is a sufficiently large constant) one by one in  $\log((1/\delta) \cdot \log \log n)$  rounds, which increases the available memory (compared to the number of (remaining) nodes) by a factor of  $\Omega((1/\delta) \cdot \log \log n)$ , by Lemma 4.3. Since the algorithm for constructing the  $H$ -partition consist of  $O((1/\delta) \cdot \log \log n)$  iterations, this implies that we can store all edges that we add during the further course of the construction simultaneously without violating

<sup>7</sup>Note that it is easy to keep track of which edges are original and which are added, incurring only a small constant memory overhead; later we will also argue why storing the added edges does not violate our memory constraints.



the memory restriction, by an argument similar to the analogous statement for the old construction of the  $H$ -partition. Similarly, also the number of added edges incident to one particular node does not exceed the memory capacity of a single machine. In the following, we assume that this pre-processing step took place and all edges added during the construction of the  $H$ -partition are also available for the simulation.

**Matching Algorithm:** As mentioned above, we process the chunks one by one, in decreasing order w.r.t. the indices of the contained layers. After processing a chunk, we want each node contained in the chunk to know the output of all incident edges according to the centralized matching algorithm. In the following, we describe how to process a chunk, after some preliminary “global” steps.

The mark-and-propose phase of the algorithm is straightforward to implement in the low-memory MPC model: each node (in each chunk at the same time) performs the marking of an outgoing edge as specified in the algorithm description.<sup>8</sup> The proposing is performed for all nodes before going through the chunks sequentially: each node proposes one of its marked incoming edges (if there is at least one) uniformly at random. Note that proposing an edge does not necessarily indicate that this edge will be added to the matching; more specifically, an edge proposed by some node  $v$  will be added to the matching iff the edge that  $v$  marked is not selected to be added to the matching.<sup>9</sup>

After this mark-and-propose phase, the processing of the chunks begins. Consider an arbitrary chunk. Let  $i$  be the iteration (in some phase) in which this chunk was removed in the construction of the  $H$ -partition, i.e., the chunk consists of  $2^i$  layers. Each node in the chunk collects the topology of its  $2^i$ -hop neighborhood in the chunk including the information contained therein about proposed edges. Due to the edges added during the construction of the  $H$ -partition, this can be achieved in a constant number of rounds, and by an analogous argument to the one at the end of Section 4.2, collecting the indicated information does not violate the memory restrictions of our model. Lemma 4.5 shows that the information contained in the  $2^i$ -hop neighborhood of a node is sufficient for the node to determine the output for each incident edge in the centralized matching algorithm.

**Lemma 4.5.** *The information about which edges are proposed in the  $2^i$ -hop neighborhood of a node  $v$  uniquely determines the output of all edges incident to  $v$  according to the centralized matching algorithm.*

**PROOF.** From the design of the centralized matching algorithm, it follows that an edge is part of the matching iff 1) the edge is proposed and 2) either the higher-layer endpoint of the edge has no outgoing edges or the outgoing edge marked by the higher-layer endpoint is not part of the matching. Hence, in order to check whether an incident edge is in the matching, node  $v$  only has to consider the unique directed chain of proposed edges (in the chunk) starting in  $v$ . Clearly, the information which of the edges in this

chain are proposed uniquely defines the output of the first edge in the chain, from which  $v$  can infer the output of all other incident edges. Since the number of edges in the chain is bounded by  $2^i - 1$  as the chain is directed, the lemma statement follows.  $\square$

It thus follows from the bound on the number of iterations that the simulation of the selection phase for the matching algorithm can be performed in  $O(1/\delta \cdot \log \log n)$  rounds of communication.

**Independent Set Algorithm:** The simulation of the independent set algorithm proceeds analogously to the case of the matching algorithm. First, each node performs the marking and proposing in a distributed fashion in a constant number of rounds. Then, the chunks are processed one by one, as above, where during the processing of a chunk removed in iteration  $i$ , each node contained in the chunk collects its  $2^i$ -hop neighborhood, including the information about which nodes are proposed, and then computes its own output locally. By analogous arguments to the ones presented in the case of the matching algorithm, the algorithm adheres to the memory constraints of our model and the total number of communication rounds is  $O(1/\delta \cdot \log \log n)$ . The only part of the argumentation where a bit of care is required is the analogue of Lemma 4.5: In the case of the independent set algorithm the output of a node  $v$  may depend on *each* of its parents since each of those could be part of the independent set, which would prevent  $v$  from joining the independent set. However, *all* nodes in the chunk that can be reached from  $v$  via a directed chain of edges are contained in  $v$ 's  $2^i$ -hop neighborhood; therefore, collecting the own  $2^i$ -hop neighborhood is sufficient for determining one's output. Note that at the end of processing a chunk, if we follow the above implementation, we have to spend an extra round for removing the neighbors of all selected independent set nodes since these may be contained in another chunk.

## 5 GENERAL ALGORITHMIC TOOLS FOR LOW-MEMORY MPC

In this section, we introduce a set of algorithmic tools that yield convenient basic primitives in the low-memory MPC model. Some variant of each of the tools is used in our work and hence, we believe that the more generic versions provided below may prove helpful in the future work in the area. We note that our variants have some problem-specific fine-tuning and hence, the guarantees given in the statements that follow are slightly weaker than the ones proven for our MIS and MM algorithms.

### 5.1 Graph Exponentiation via All-to-All Communication

The goal of the *graph exponentiation* technique is to collect the  $t$ -hop of every node in  $O(\log t)$  rounds, i.e., create the  $t$ -th power  $G^t$  of the input graph  $G$  [23]. The approach is to inductively collect the  $2^i$ -hop of every node by round  $i$  so long as we are guaranteed that the size of each of them is sufficiently small.

**Lemma 5.1.** *Given that for any node  $v$ , size of its  $t$ -hop is bounded by  $n^\beta$  for any  $\beta \leq \delta/2$ , there exists an algorithm that gathers the  $t$ -hop of every node in at least one machine within at most  $O(\log t)$*

<sup>8</sup>Note that, formally, the algorithm for construction the  $H$ -partition only returns the layer index for each node; however, from this information each node can easily determine which edges are outgoing, unoriented, or incoming according to the partial orientation induced by the  $H$ -partition.

<sup>9</sup>In other words, only proposed edges can go into the matching and whether such an edge indeed goes into the matching can be determined by going through the layers in decreasing order and only adding a proposed edge if there is no conflict.



rounds of MPC using  $O(n^\delta)$  space per machine and  $O(n^{1+2\beta})$  total space.

## 5.2 Load Balancing

This section addresses one of the technical details faced by low-memory MPC graph algorithms. Assume that we are given a separable function (cf. Definition 4.1) which we need to compute for all the nodes in the graph. For any node  $v$  the value of this function is based on its neighbors. The complication here arises from the fact that the degree of the nodes can be larger than the memory of the machines. Therefore, we are not able to simply gather neighbors of  $v$  in one machine and compute the given function. Examples of such functions are as follows.

- Finding the degree of the nodes.
- For any node in the graph find the neighbor with the minimum label. One usage of this is in the Luby's algorithm when each node picks a random number and we need to find the nodes who have the minimum number among their neighbors.
- Some nodes of the graph are chosen to be in the MIS and each node needs to know whether it is adjacent to any such node or not.

It is easy to verify that all of the functions above are separable. In Lemma 5.2 we prove that there is an algorithm that takes  $O(1/\delta)$  rounds of MPC using  $O(n^\delta)$  space per machine and  $\tilde{O}(m)$  total space. For space limits, we omit the proof.

**Lemma 5.2.** *Fix an arbitrary  $n$ -vertex graph and suppose that we have a number  $x_v$  for any vertex  $v$  stored in some arbitrary machine. Given an arbitrary separable function  $f$ , there is an MPC algorithm that outputs, for every vertex  $v$ , the value of  $f(\{x_{u_1}, \dots, x_{u_d}\})$  where  $u_1, \dots, u_d$  are the vertices adjacent to  $v$ . If the space per machine is  $O(n^\delta)$ , the algorithm takes  $O(1/\delta)$  rounds and uses  $\tilde{O}(m)$  total space.*

## 5.3 Blind Coordination

The goal of this section is to highlight the simple but powerful concept of *blind coordination*. We apply this technique to compress multiple rounds of a large class of LOCAL algorithms, which we call *state-congested* local algorithms. Roughly speaking, in a state-congested local algorithm, we can maintain *states* on the nodes/edges over the rounds of the algorithm, but we restrict these states to be of size  $O(\log n)$  bits and depend (loosely speaking) only on the states of the 1-hop of every node/edge at the previous round.

**Definition 5.3.** *A distributed LOCAL algorithm is state-congested if:*

- (1) *By the end of each round  $r$ , on any node  $v$  (and respectively on any edge  $e$ ), the algorithm stores a state  $s_r(v)$  (resp.  $s_r(e)$ ) of size  $O(\log n)$  bits. The initial state  $s_0(v)$  of each node  $v$  is its ID and the initial state  $s_0(e)$  of each edge  $e$  is the IDs of its two endpoints.*
- (2) *The state  $s_r(v)$  of each node  $v$  by the end of any round  $r$ , depends only on its state  $s_{r-1}(v)$  in the previous round, the states of its incident edges  $\{s_{r-1}(e) \mid e \ni v\}$  in the previous round, and its tape  $p(v)$  of poly  $\log n$  random bits. Furthermore,*

*the state  $s_r(e)$  of each edge  $e = (u, v)$  by the end of round  $r$  is only a function of  $s_{r-1}(u)$ ,  $s_{r-1}(v)$  and  $s_{r-1}(e)$ .*

- (3) *The states of the nodes/edges at the last round of the algorithm are sufficient in determining, collectively, the output of the algorithm.*

The key property of state-congested local algorithms is that the intermediate states of the algorithm are also small. This is in contrast, for example, with algorithms in which each node  $v$  first collects its, say,  $O(\log n)$ -hop and then makes its final decision in one shot. We note that state-congested local algorithms are similar to, but more restrictive, than a variant of LOCAL algorithms called CONGEST (see [30]) where the messages over the links are restricted to have  $O(\log n)$  bits. Before describing the main result of this section, we need another definition.

**Definition 5.4.** *We call a state-congested local algorithm low-memory, if updating the state each node  $v$  can be done in a space of size  $O(\deg(v) \cdot \text{poly} \log n)$  bits and updating the state of each edge  $e$  requires a space of size  $O(\text{poly} \log n)$  bits.*

The definition above is required to ensure, e.g., that once we have the states and random tapes of all neighbors of a node  $v$ , we can update the state of  $v$  without using any extra space. This is almost always satisfied. We are now ready to formalize the main lemma of this section which results in compressing  $r$  rounds of a state-congested local algorithm into much fewer rounds of a low-memory MPC algorithm if the maximum degree  $\Delta$  of the graph is small. The main theorem is as follows:

**Lemma 5.5.** *For any graph with  $n$  nodes,  $m$  edges, and maximum degree  $\Delta \leq n^\delta$ , where  $\delta$  is a desirably small constant number in  $(0, 1)$ , one can compress  $r$  rounds of any low-memory state-congested local algorithm in  $O(\frac{r}{\log_\Delta n} + \log \log_\Delta n)$  rounds of MPC using  $O(n^\delta)$  space per machine and  $\tilde{O}(m + n^{1+2\delta/3})$  total memory.*

**PROOF.** Suppose that our goal is to compress  $r$  rounds of a low-memory state-congested local algorithm  $\mathcal{A}$ . Initially, each node will be assigned to a machine that will be responsible for keeping track of its state. Note that since the total space is at least  $n^{1+2\delta/3}$ , and memory per machine is  $O(n^\delta)$ , we have at least  $\Omega(n^{1-\delta/3})$  machines. It suffices to make each machine responsible for  $n^{2\delta/3}$  nodes. This assignment can easily be done based on, say, the nodes' IDs. Let  $t := \lfloor \frac{\delta}{3} \log_\Delta n \rfloor$ . We first collect the  $t$ -hop neighborhood of each node in the machine that is responsible for it. Note that  $t$  is chosen to be small enough that the  $t$ -hop neighborhood of every node has at most  $\Delta^t \leq n^{\delta/3}$  edges which is substantially smaller than the memory per machine. Therefore, we can use the graph exponentiation algorithm of Lemma 5.1 to collect the  $t$ -hop of every node in the machine responsible for it in only  $O(\log t) = O(\log \log_\Delta n)$  rounds using a total memory of size  $\tilde{O}(m + n^{1+2\delta/3})$ .

After collecting the neighborhoods, on each machine, we run  $t$  rounds of  $\mathcal{A}$  on the subgraph that is stored in it. This can be done in only one round of MPC since no communication between the machines is required. The main intuition behind the *blind-coordination* idea is that the final state of a node  $v$  in the machine that is responsible for it is exactly the same as that of  $v$  after  $t$  rounds of the original algorithm  $\mathcal{A}$ . We emphasize that a node  $v$  may also be stored in machines not responsible for  $v$ , and in fact,

the states computed for  $v$  in those machines might be completely different from its *correct* state by the end of round  $t$  of  $\mathcal{A}$ . However, crucially, the state of each node matches its correct state in the machine responsible for it. Formally, let us denote by  $s_i(\cdot)$  the state of a node or an edge by the end of round  $i$  of algorithm  $\mathcal{A}$  and denote by  $\hat{s}_{\mu,i}(\cdot)$ , for any  $i \leq t$ , the state of a node or an edge after simulating  $i$  rounds of  $\mathcal{A}$  on the subgraph stored in machine  $\mu$ ; we have:

**Observation 5.6.** *Let  $v$  denote an arbitrary node whose  $i$ -hop neighborhood is stored in machine  $\mu$  and let  $\beta$  be an arbitrary non-negative integer. If for any node or edge  $x$  in the  $i$ -hop neighborhood of  $v$  we have its correct state by round  $\beta$ , i.e.,  $\hat{s}_{\mu,\beta}(x) = s_\beta(x)$ , then we compute the correct state of  $v$  after  $\beta + i$  rounds in machine  $\mu$ , i.e.,  $\hat{s}_{\mu,\beta+i}(v) = s_{\beta+i}(v)$  without any rounds of communication. Similarly, for any edge  $e$  incident to  $v$ , we have  $\hat{s}_{\mu,\beta+i}(e) = s_{\beta+i}(e)$ .*

Recall that our goal was to compress  $r$  rounds of a low-memory state-congested algorithm  $\mathcal{A}$  into few rounds of a low-memory MPC algorithm. By the discussion above, after collecting the  $t$ -hop neighborhood of every node in  $O(\log \log_\Delta n)$  rounds, if  $r < t$ , then the simulation takes only  $O(1)$  extra rounds to complete. For most applications, however,  $r$  is much larger than  $t$ . In such cases, we cannot afford to collect the  $r$ -hop neighborhood of a node in one machine as its size may exceed the space per machine. The idea, here, is to compress every  $t$  rounds of  $\mathcal{A}$  into  $O(1)$  rounds of our low-memory MPC algorithm. To do this, with the above-mentioned approach we can access the state of each node and edge after  $t$  rounds. The idea, then, is that each machine shares the states of the nodes/edges that it is responsible for with other machines. Once collected this information, each machine then runs  $\mathcal{A}$  for another  $t$  rounds with respect to the now updated states. By the end of this round, we are aware of the state  $s_{2t}(\cdot)$  of each node/edge on its responsible machine. We can continue this process for  $r/t$  rounds to complete compression of  $r$  rounds of  $\mathcal{A}$ . Overall it takes only  $O(\frac{r}{t} + \log t) = O\left(\frac{r}{\epsilon \log_\Delta n} + \log \log_\Delta n\right)$  rounds to have the states of all nodes/edges by the end of round  $r$ .  $\square$

## REFERENCES

- [1] Alexandr Andoni, Aleksandar Nikolov, Krzysztof Onak, and Grigory Yaroslavtsev. 2014. Parallel Algorithms for Geometric Graph Problems. In *Proceedings of the Symposium on Theory of Computing (STOC)*. 574–583.
- [2] Sepehr Assadi, MohammadHossein Bateni, Aaron Bernstein, Vahab Mirrokni, and Cliff Stein. 2017. Coresets Meet EDCS: Algorithms for Matching and Vertex Cover on Massive Graphs. *arXiv preprint: 1711.03076* (2017).
- [3] Leonid Barenboim and Michael Elkin. 2010. Sublogarithmic Distributed MIS Algorithm for Sparse Graphs Using Nash-Williams Decomposition. *Distributed Computing* 22, 5-6 (2010), 363–379.
- [4] Leonid Barenboim, Michael Elkin, Seth Pettie, and Johannes Schneider. 2016. The Locality of Distributed Symmetry Breaking. *Journal of the ACM (JACM)* 63, 3 (2016), 20.
- [5] MohammadHossein Bateni, Soheil Behnezhad, Mahsa Derakhshan, Mohammad-Taghi Hajiaghayi, Raimondas Kiveris, Silvio Lattanzi, and Vahab Mirrokni. 2017. Affinity Clustering: Hierarchical Clustering at Scale. In *the Proceedings of the Conference on Neural Information Processing Systems (NIPS)*. 6867–6877. <http://papers.nips.cc/paper/7262-affinity-clustering-hierarchical-clustering-at-scale.pdf>
- [6] Paul Beame, Paraschos Koutiris, and Dan Suciu. 2014. Skew in Parallel Query Processing. In *the Proceedings of the 33rd ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*. 212–223.
- [7] Paul Beame, Paraschos Koutiris, and Dan Suciu. 2017. Communication Steps for Parallel Query Processing. *Journal of the ACM (JACM)* 64, 6 (2017), 40.
- [8] Sebastian Brandt, Manuela Fischer, and Jara Uitto. 2019. Breaking The Linear-Memory Barrier in MPC: Fast MIS on Trees with Strongly Sublinear Memory. In *the Proceedings of the International Colloquium on Structural Information and Communication Complexity (SIROCCO)*. to appear.
- [9] Graham Cormode, Hossein Jowhari, Morteza Monemizadeh, and S. Muthukrishnan. 2017. The Sparse Awakens: Streaming Algorithms for Matching Size Estimation in Sparse Graphs. In *the Proceedings of the Annual European Symposium on Algorithms (ESA)*. 29:1–29:15.
- [10] Artur Czumaj, Jakub Łącki, Aleksander Mądry, Slobodan Mitrović, Krzysztof Onak, and Piotr Sankowski. 2018. Round Compression for Parallel Matching Algorithms. In *Proceedings of the Symposium on Theory of Computing (STOC)*. 471–484.
- [11] Jeffrey Dean and Sanjay Ghemawat. 2008. MapReduce: Simplified Data Processing on Large Clusters. *Commun. ACM* 51, 1 (2008), 107–113.
- [12] Hossein Esfandiari, Mohammad Taghi Hajiaghayi, Vahid Liaghat, Morteza Monemizadeh, and Krzysztof Onak. 2015. Streaming Algorithms for Estimating the Matching Size in Planar Graphs and Beyond. In *the Proceedings of ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 1217–1233.
- [13] Mohsen Ghaffari. 2016. An Improved Distributed Algorithm for Maximal Independent Set. In *the Proceedings of ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 270–277.
- [14] Mohsen Ghaffari, Themis Gouleakis, Christian Konrad, Slobodan Mitrović, and Ronitt Rubinfeld. 2018. Improved Massively Parallel Computation Algorithms for MIS, Matching, and Vertex Cover. In *Proceedings of the International Symposium on Principles of Distributed Computing (PODC)*. 129–138.
- [15] Mohsen Ghaffari and Jara Uitto. 2019. Sparsifying Distributed Algorithms with Ramifications in Massively Parallel Computation and Centralized Local Computation. In *the Proceedings of ACM-SIAM Symposium on Discrete Algorithms (SODA)*.
- [16] Gaurav Goel and Jens Gustedt. 2006. Bounded Arboricity to Determine the Local Structure of Sparse Graphs. In *International Workshop on Graph-Theoretic Concepts in Computer Science*. Springer, 159–167.
- [17] Michael T. Goodrich, Nodari Sitchinava, and Qin Zhang. 2011. Sorting, Searching, and Simulation in the MapReduce Framework. 374–383.
- [18] Elena Grigorescu, Morteza Monemizadeh, and Samson Zhou. 2016. Estimating Weighted Matchings in  $o(n)$  Space. *CoRR abs/1604.07467* (2016). [arXiv:1604.07467](http://arxiv.org/abs/1604.07467)
- [19] Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. 2007. Dryad: Distributed Data-Parallel Programs from Sequential Building Blocks, Vol. 41. 59–72.
- [20] Howard Karloff, Siddharth Suri, and Sergei Vassilvitskii. 2010. A Model of Computation for MapReduce. In *the Proceedings of ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 938–948.
- [21] Amos Korman, Jean-Sébastien Sereni, and Laurent Viennot. 2013. Toward More Localized Local Algorithms: Removing Assumptions Concerning Global Knowledge. *Distributed Computing* 26, 5-6 (2013), 289–308.
- [22] Silvio Lattanzi, Benjamin Moseley, Siddharth Suri, and Sergei Vassilvitskii. 2011. Filtering: a Method for Solving Graph Problems in MapReduce. In *the Proceedings of the Symposium on Parallel Algorithms and Architectures (SPAA)*. 85–94.
- [23] Christoph Lenzen and Roger Wattenhofer. 2010. Brief Announcement: Exponential Speed-Up of Local Algorithms Using Non-Local Communication. In *Proceedings of the International Symposium on Principles of Distributed Computing (PODC)*. 295–296.
- [24] Nathan Linial. 1992. Locality in Distributed Graph Algorithms. *SIAM J. Comput.* 21, 1 (1992), 193–201.
- [25] Zvi Lotker, Boaz Patt-Shamir, and Seth Pettie. 2015. Improved Distributed Approximate Matching. *Journal of the ACM (JACM)* 62, 5 (2015).
- [26] Andrew McGregor. 2005. Finding Graph Matchings in Data Streams. In *Approximation, Randomization and Combinatorial Optimization. Algorithms and Techniques*. Springer, 170–181.
- [27] Crispin St. J. Nash-Williams. 1961. Edge-Disjoint Spanning Trees of Finite Graphs. *Journal of the London Mathematical Society* 1, 1 (1961), 445–450.
- [28] Crispin St. J. Nash-Williams. 1964. Decomposition of Finite Graphs into Forests. *Journal of the London Mathematical Society* 1, 1 (1964), 12–12.
- [29] Krzysztof Onak, Baruch Schieber, Shay Solomon, and Nicole Wein. 2018. Fully Dynamic MIS in Uniformly Sparse Graphs. In *the Proceedings of the International Colloquium on Automata, Languages and Programming (ICALP)*. 92:1–92:14.
- [30] David Peleg. 2000. *Distributed Computing: A Locality-Sensitive Approach*. Society for Industrial and Applied Mathematics.
- [31] Sriram V. Pemmaraju. 2001. Equitable Coloring Extends Chernoff-Hoeffding Bounds. In *Approximation, Randomization, and Combinatorial Optimization: Algorithms and Techniques*. Springer, 285–296.
- [32] Tom White. 2012. *Hadoop: The Definitive Guide*.
- [33] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2010. Spark: Cluster Computing with Working Sets. *HotCloud* 10 (2010), 95.